There's nothing special about binding as the receiver of the pry message. It's handy because it's available anywhere and gives you the entire local scope—and Pry will display the surrounding code when you drop into it. You can, however, send pry to anything. You could've used creator.pry in the same test, and you would've been dropped into Pry with the creator local variable as the top-level scope.

The pry-byebug and pry-stack_explorer gems have some commands that are useful in a debugging session.

From pry-byebug you get four commands: continue, finish, next, and step. If you've done much with interactive debuggers, these will be familiar. The step and next commands take you one command forward, with next keeping you in the same frame but step sending you into a frame. The finish command executes until the end of the current frame, and continue ends the Pry session entirely.

The pry-stack_explorer gem allows you to see the entire current stack trace with the show-stack command. You can then use the up and down methods to navigate the trace.

You can also invoke the editor directly from the Pry console using the edit command. Using just edit -c opens the editor to the file where the original binding.pry was located. Using edit ClassName#method_name opens the editor at the file where the given class and method are defined. Similarly, edit with any instance variable as an argument opens the file where that instance's class is defined.

The Pry session will wait while the edit is happening. Closing the edit window will reload that file in the Pry session and allow you to keep going.

You can be even more interactive with the pry-rescue gem. The pry-rescue gem provides a regular terminal command rescue that is prefixed to a different command. If that command raises an exception, then Pry is automatically invoked at the point of the exception. If you run rescue before your test command, Pry will also be invoked on test failure.

In RSpec, pry-rescue just works with a rescue rspec command. In Minitest you need to put the following lines in the test_helper.rb first:

```
require "minitest/autorun"
require "pry-rescue/minitest"
```

Remember how I said that editing files from Pry causes the file to be reloaded? Wouldn't it be great if you could rerun the failing test inside Pry? The command for doing just that is try-again.

Using these tools together can give you a very interactive workflow. Run the tests via rescue, drop into Pry on failure, keep editing and trying again until